

DESIGNING A PRIVACY-PRESERVING, OFFLINE MACHINE LEARNING SYSTEM
FOR MALICIOUS URL DETECTION USING URL FEATURES

BY

CONNIE RODRIGUEZ

AN ADVANCED DESIGN PROJECT SUBMITTED TO THE FACULTY OF SAINT
MARTIN'S UNIVERSITY IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR
THE DEGREE OF MASTER OF SCIENCE IN COMPUTER SCIENCE

SAINT MARTIN'S UNIVERSITY

MAY 7, 2026

CSC 598 CERTIFICATE OF APPROVAL

DESIGNING A PRIVACY-PRESERVING, OFFLINE MACHINE LEARNING SYSTEM
FOR MALICIOUS URL DETECTION USING URL FEATURES

BY

CONNIE RODRIGUEZ

Date of Acceptance

Special Committee directing the
Master's degree for Candidate

Dr. Dvorak, Professor
Project Adviser

Professor: _____
Committee Member

Professor: _____
Ex-Officio

Abstract

This research presents the design, implementation, and evaluation of a privacy-preserving, offline machine learning system for malicious URL detection using URL-based features. Unlike many current malicious URL detection approaches that rely on external data sources such as Domain Name System (DNS) queries, WHOIS records, or webpage content analysis, this system operates solely on features derived from the URL string. This constraint enables deployment in air-gapped (i.e., isolated from external networks), privacy-sensitive, and resource-constrained environments, while supporting reproducibility and eliminating dependencies on external services. This design directly targets a gap in existing literature, where offline, privacy-preserving detection frameworks remain underexplored relative to their practical deployment need in isolated and constrained environments.

The system is implemented as a research-oriented prototype using a modular, multi-stage pipeline. The pipeline performs dataset construction, preprocessing, feature engineering (deriving lexical and structural attributes from URLs), model training, and evaluation using two real-world threat intelligence sources: PhishTank, a community verified repository of phishing URLs, and URLhaus, a curated feed of malware distribution URLs, balanced against legitimate domains from the Tranco Top Sites list. A total of 24 lexical and structural URL features are engineered and applied consistently across datasets to ensure methodological integrity and prevent data leakage. Three lightweight machine learning models, Logistic Regression, Decision Tree, and Random Forest, are trained and evaluated using stratified sampling and five-fold cross-validation.

A central contribution of this work is the evaluation of cross-dataset generalization, where models trained on one threat intelligence source are tested on another to assess their ability to maintain performance across different data sources and real-world variability. Additionally, the system introduces a single-responsibility pipeline architecture, enhancing transparency, reproducibility, and extensibility for academic and research applications.

This study contributes to cybersecurity research by analyzing the trade-offs between detection accuracy, generalization capability, computational efficiency, and privacy preservation in a fully offline detection framework. The evaluation addresses the viability of URL-only feature-based detection while examining the limitations and challenges associated with operating under strict data and environmental constraints. This work establishes a reproducible framework for future research and offers insights relevant to the potential deployment of lightweight malicious URL detection systems in constrained environments.

List of Figures and Tables

List of Figures

Figure 1 End-to-End Pipeline Diagram- Malicious URL Detection System	26
Figure 2 Within-Dataset Evaluation Dashboard: PhishTank and Tranco Dataset	62
Figure 3 Within-Dataset Evaluation Dashboard: URLhaus and Tranco Dataset	62
Figure 4 Cross-Dataset Evaluation Dashboard: Train on PhishTank, Test on URLhaus	65
Figure 5 Cross-Dataset Evaluation Dashboard: Train on URLhaus, Test on PhishTank	66
Figure 6 Feature Selection Impact Dashboard: Within-Dataset Evaluation	69
Figure 7 Feature Selection Impact Dashboard: Cross-Dataset Evaluation	70
Figure 8 Hyperparameter Tuning Three-Way Comparison Dashboard: Within-Dataset Evaluation.....	74
Figure 9 Hyperparameter Tuning Three-Way Comparison Dashboard: Cross-Dataset Evaluation.....	75
Figure 10 XML Parsing.....	94
Figure 11 URL Extraction.....	95
Figure 12 Verification Filter	95
Figure 13 DataFrame and Deduplication	95
Figure 14 CSV Output.....	96
Figure 15 CLI and Entry Point	96
Figure 16 URLhaus Column Schema	97
Figure 17 File Parsing	97
Figure 18 Encoding Fallback	97
Figure 19 URL Cleaning	98
Figure 20 Deduplicating and Labeling.....	98
Figure 21 CSV Output.....	98
Figure 22 CLI and Entry Point	99
Figure 23 PhishTank Loading	100
Figure 24 URLhaus Loading.....	100
Figure 25 Tranco Loading.....	101
Figure 26 Class Balancing.....	102
Figure 27 Dataset Combination	102
Figure 28 Deterministic Shuffle and Output.....	103
Figure 29 Dataset Generation Workflow	103
Figure 30 Dataset Defaults	104
Figure 31 Label Mapping.....	104
Figure 32 CSV Loading	105
Figure 33 Column Standardization	105

Figure 34 Column Rename	105
Figure 35 Label Normalization	106
Figure 36 URL Cleaning	106
Figure 37 Label Validation	106
Figure 38 Duplicate Removal	107
Figure 39 Class Balance Logging	107
Figure 40 Cleaning Workflow	107
Figure 41 Dataset Defaults	108
Figure 42 CSV Loading	108
Figure 43 Entropy Calculation	109
Figure 44 URL Parsing	109
Figure 45 Length Features	109
Figure 46 Character Count Features	110
Figure 47 Structural Features	110
Figure 48 Subdomain Depth	110
Figure 49 Entropy and Ratio Features	111
Figure 50 Output Column Structure	111
Figure 51 Feature File Output	112
Figure 52 Feature File Inputs	112
Figure 53 Training Log Setup	113
Figure 54 Label Normalization	113
Figure 55 Feature and Label Preparation	114
Figure 56 Stratified Train-Test Split	114
Figure 57 Model Evaluation Function	115
Figure 58 Logistic Regression Training	115
Figure 59 Random Forest Training	116
Figure 60 Feature Importance Extraction	116
Figure 61 Decision Tree Training	116
Figure 62 Model Artifact Saving	117
Figure 63 Cross-Validation	117
Figure 64 Dashboard Generation	117
Figure 65 Results Output	118
Figure 66 Cross-Evaluation Inputs	118
Figure 67 Dataset Preparation	118
Figure 68 Cross-Evaluation Direction	119
Figure 69 Model Evaluation Function	119
Figure 70 Logistic Regression Scaling	120
Figure 71 Random Forest Evaluation	120

Figure 72 Feature Importance Extraction	120
Figure 73 Decision Tree Evaluation	121
Figure 74 Dashboard Output	121
Figure 75 Two-Direction Evaluation Workflow	122
Figure 76 Results Export	122
Figure 77 Model Artifact Paths	123
Figure 78 Single-URL Entropy	123
Figure 79 URL Parsing	123
Figure 80 Single-URL Feature Extraction	124
Figure 81 Entropy and Ratio Features	124
Figure 82 Feature DataFrame Assembly	125
Figure 83 Model Loading	126
Figure 84 Inference Preprocessing	126
Figure 85 Model Predictions	127
Figure 86 Majority Vote	127
Figure 87 Result Formatting	127
Figure 88 CLI and Entry Point	128
Figure 89 Iteration 2 Inputs	128
Figure 90 Dataset Preparation	129
Figure 91 Random Forest Importance Setup	129
Figure 92 Feature Importance Extraction	129
Figure 93 Feature Selection Threshold	130
Figure 94 Model Evaluation	130
Figure 95 Result Dictionary	131
Figure 96 Column Alignment	131
Figure 97 Feature Selection Execution	131
Figure 98 Within-Dataset Comparison	132
Figure 99 Cross-Dataset Comparison	133
Figure 100 Results and Importance Export	133
Figure 101 Reduced Feature Set	133
Figure 102 Parameter Grids	134
Figure 103 Baseline Comparison Data	134
Figure 104 Dataset Preparation	135
Figure 105 Grid Search Setup	135
Figure 106 Logistic Regression Tuning	135
Figure 107 Random Forest Tuning	136
Figure 108 Decision Tree Tuning	136
Figure 109 Tuned Model Evaluation	136

Figure 110 Tuned Result Record	137
Figure 111 Reduced Feature Validation	137
Figure 112 Within-Dataset Tuning Workflow.....	138
Figure 113 Cross-Dataset Tuning Workflow	138
Figure 114 Results and Parameters Export.....	139
Figure 115 Dashboard Generation.....	139
Figure 116 GUI Interface	140
Figure 117 Model Artifact Paths	140
Figure 118 GUI Color Scheme.....	141
Figure 119 GUI Feature Extraction	142
Figure 120 Feature DataFrame Assembly.....	142
Figure 121 Model Loading.....	142
Figure 122 GUI Window Initialization	143
Figure 123 Input Controls	144
Figure 124 Results Panel	144
Figure 125 GUI Classification Logic.....	145
Figure 126 Result Update Logic	146
Figure 127 GUI Entry Point	147
Figure 128 UML Use Case Diagram- Current Prototype	148
Figure 129 UML Use Case Diagram- Future Deployment.....	149
Figure 130 UML Activity Diagram- Training Pipeline and Inference Path	150
Figure 131 UML Sequence Diagram- End-to-End Pipeline Interaction	151
Figure 132 UML Class Diagram- System Architecture	153
Figure 133 UML State Diagram- Training Pipeline.....	155
Figure 134 UML State Diagram- Inference Path.....	156
Figure 135 Entity-Relationship Diagram- Pipeline Data Flow	157

List of Tables

Table 1 <i>Eight-Script Pipeline Architecture: Scripts, Responsibilities, Inputs, and Outputs</i>	18
Table 2 Within-Dataset Performance Results: PhishTank and Tranco Dataset.....	58
Table 3 Top 10 Feature Importances: PhishTank Training	59
Table 4 Within-Dataset Performance Results: URLhaus and Tranco Dataset	60
Table 5 Top 10 Feature Importances: URLhaus Training.....	60
Table 6 Cross-Dataset Generalization Results: Train on PhishTank, Test on URLhaus	63
Table 7 Cross-Dataset Generalization Results: Train on URLhaus, Test on PhishTank	63
Table 8 Within-Dataset Performance Comparison: Full 24 vs. Reduced 13 Features...	67

Table 9	Cross-Dataset Performance Comparison: Full 24 vs. Reduced 13 Features ...	68
Table 10	Best Hyperparameter Configurations from Grid Search	71
Table 11	Within-Dataset Performance: Three-Way Comparison	72
Table 12	Cross-Dataset Performance: Three-Way Comparison	72
Table 13	Feature Importance Scores and Stability Classification.....	76
Table 14	Final Locked Hyperparameter Configurations.....	81

Table of Contents

Abstract.....	3
List of Figures and Tables	4
CHAPTER 1: INTRODUCTION.....	12
1.1 Introduction.....	12
1.2 Background.....	14
2.1 Project Description	16
2.1.1 Pipeline Architecture	17
2.1.2 Project Objectives	19
2.2 Analysis.....	20
2.2.1 Rationale for Lightweight Lexical Features	20
2.2.2 Model Selection Analysis	21
2.2.3 Dataset Selection Strategy	21
2.2.4 Expected Challenges.....	22
2.2.5 Analysis of Project Scope	22
2.2.6 Transition to Advanced Design	23
2.2.7 Iterative Experimental Methodology	23
CHAPTER 3: SYSTEM DESIGN.....	25
3.1 System Design Principles	25
3.2 System Architecture	25
3.3 Data Preparation and Preprocessing	28
3.3.1 Encoding and File Handling	28
3.3.2 Schema Validation	28
3.3.3 Duplicate Removal	29
3.3.4 Invalid Entry Removal	29
3.3.5 Label Normalization.....	29
3.3.6 Cleaned Dataset Output	29
3.4 Feature Engineering Approach	29
3.4.1 Group A- Length Measurements	30

3.4.2 Group B- Character Count Features	30
3.4.3 Group C- Structural and Boolean Features	31
3.4.4 Group D- Entropy and Ratio Features	31
3.4.5 Feature Dataset Output	32
3.4.6 Design Rationale and Limitations	32
3.5 Model Design and Rationale.....	33
3.5.1 Logistic Regression	33
3.5.2 Random Forest	34
3.5.3 Decision Tree	35
3.6 Experimental Setup	36
3.6.1 Baseline Experimental Setup (Iteration 1)	36
3.6.2 Feature Selection Methodology (Iteration 2)	39
3.6.3 Hyperparameter Tuning Methodology (Iteration 3).....	41
3.6.4 Model Training	44
3.6.5 Evaluation Metrics	47
3.7 Cross-Dataset Generalization Procedures.....	49
3.8 Design Justification	53
3.9 Summary	57
CHAPTER 4: RESULTS AND FINDINGS.....	58
4.1 Within-Dataset Performance (Iteration 1 Baseline).....	58
4.2 Cross-Dataset Generalization.....	62
4.3 Feature Selection Results (Iteration 2)	66
4.4 Hyperparameter Tuning Results (Iteration 3)	70
4.5 Feature Stability Analysis	75
4.6 Final Model Selection and System Lock-in	78
4.7 Interpretation of Results	81
4.8 Summary of Findings	83
CHAPTER 5: CONCLUSION AND DISCUSSION	85
5.1 Conclusion	85

5.2 Discussion	86
5.2.1 The Viability and Limitations of URL-String-Only Detection	86
5.2.2 The Value of the Iterative Experimental Methodology	86
5.2.3 Model Selection Trade-offs for Deployment	87
5.2.4 Relationship to Existing Literature	87
5.3 Future Work	88
5.3.1 Feature Representation and Generalization Enhancement	88
5.3.2 Multi-Source Training and Ensemble Optimization	88
5.3.3 Temporal Evaluation and Practical Deployment.....	88
APPENDIX I: AI Use.....	91
APPENDIX II: System Pipeline and Code	94
APPENDIX III: System Architecture and Design Diagrams.....	148
APPENDIX IV: Source Code and Supporting Materials	158

CHAPTER 1: INTRODUCTION

1.1 Introduction

Phishing remains one of the most persistent and damaging forms of cybercrime, responsible for credential theft, financial fraud, identity compromise, and the initial access phase of countless other attacks [1]. By disguising malicious links as legitimate ones, attackers exploit human trust rather than software vulnerabilities, making phishing both technically simple and socially effective. Global cybersecurity reports consistently rank phishing as the leading initial attack vector, with millions of malicious URLs generated daily and increasingly sophisticated techniques appearing across the web, mobile, and email platforms [1], [2].

Despite rising user awareness, phishing remains effective because attackers rapidly adapt URL structures, domain variations, and obfuscation techniques. Traditional defensive mechanisms, such as blacklists, signature-based filters, or heuristic rules, struggle to keep pace with these evolving threats. Blacklists can only block previously identified malicious domains; heuristics can be easily bypassed through minor variations in URL length, token placement, or character composition [3]. As a result, organizations and individuals remain vulnerable to emerging zero-day phishing URLs that do not resemble known malicious patterns.

This challenge motivates the need for adaptive, data-driven detection systems that can generalize beyond known examples. Machine learning (ML) offers this adaptability. Instead of relying on fixed rules, ML models learn statistical patterns of malicious versus legitimate URLs and can classify newly observed URLs in near real-time. URL-based detection, where the classifier evaluates only the URL text rather than webpage content, offers distinct advantages. It is fast, lightweight, does not require page rendering, and is suitable for deployment in browsers, email gateways, mobile devices, and resource-constrained environments [3].

However, a critical and underexplored constraint in practical deployment scenarios is the need to operate in a fully offline, privacy-preserving environment. Many existing malicious URL detection systems rely on supplementary inputs such as Domain Name System (DNS) queries, WHOIS registration data, webpage content inspection, or cloud-based threat intelligence feeds during classification [3]. These dependencies introduce limitations in terms of data exposure risk, reliance on network connectivity, reduced reproducibility, and limited deployability in restricted or air-gapped environments. For security tools deployed on endpoint devices, within organizational intranets, or in environments where URL resolution itself poses a risk, these external dependencies are not acceptable constraints.

This project addresses that gap by implementing a privacy-preserving, offline machine learning pipeline for malicious URL detection. The system operates exclusively on 24

lexical and structural features derived from the URL string itself, with no DNS queries, WHOIS lookups, Hypertext Transfer Protocol (HTTP) requests, or external Application Programming Interface (API) calls permitted at any stage of the pipeline. This design enables deployment in air-gapped, privacy-sensitive, and resource-constrained environments while supporting full reproducibility and eliminating runtime dependencies on external services.

The system is evaluated using two real-world threat intelligence sources: PhishTank [4], a community-verified repository of phishing URLs, and URLhaus [5], a curated feed of malware distribution URLs, balanced against legitimate domains from the Tranco Top Sites list [6]. Three lightweight machine learning classifiers, Logistic Regression, Random Forest, and Decision Tree, are trained and evaluated using stratified sampling, which preserves the proportion of malicious and legitimate URLs across training and testing sets, and five-fold cross-validation, where the data is repeatedly split into five parts to provide a more reliable estimate of model performance. A central component of evaluation is cross-dataset generalization testing, where models are trained on one threat intelligence source and evaluated on the other in both directions, assessing the system's ability to maintain detection performance across distinct real-world URL distributions.

The study seeks to answer five core research questions:

1. Can 24 URL-string-only features, derived exclusively from the URL string, without reliance on DNS queries, WHOIS lookups, or external API calls, support strong phishing detection performance across two independently sourced real-world threat intelligence feeds in a fully offline, privacy-preserving environment?
2. Which URL-string features demonstrate the highest predictive stability across the datasets from different threat intelligence sources?
3. Which lightweight machine learning algorithm provides the best trade-off between classification performance and deployment feasibility for real-world phishing detection?
4. How well does a model trained on PhishTank URLs generalize when evaluated against URLhaus data, and vice versa? What performance gap, if any, exists between the two directions, and what might it reveal about the structural differences between phishing and malware delivery URLs?
5. What measurable improvements over the Iteration 1 baseline can be demonstrated through iterative feature selection and optimization across subsequent experimental rounds?

By addressing these questions, this project makes a focused and original contribution to cybersecurity research. It presents the design, implementation, and rigorous evaluation of a fully offline, privacy-preserving malicious URL detection system that operates solely

on URL-string-derived features, without external lookups, network queries, or runtime dependencies. While existing research frequently demonstrates strong detection performance by incorporating supplementary external signals, this work deliberately excludes them to establish what is achievable within the strict constraints of air-gapped, privacy-sensitive, and resource-constrained deployment environments. This design space remains underexplored in the literature relative to its practical relevance, and this project directly addresses that gap. The resulting framework provides a reproducible, transparent, and extensible baseline for future researchers and practitioners seeking to deploy lightweight malicious URL detection in environments where system independence and data privacy are non-negotiable requirements.

1.2 Background

The field of phishing detection has evolved substantially over the past few decades. Early systems focused on manually curated blacklists and heuristic rules. While computationally inexpensive, these methods are reactive and limited. Blacklists cannot identify newly generated malicious URLs, and heuristics often produce a high false positive rate or fail entirely when attackers insert deceptive tokens, alter domain structures, or use randomized characters.

The emergence of machine learning significantly advanced the field by enabling dynamic classification based on learned patterns in URL characteristics [7]. Traditional ML approaches typically rely on handcrafted lexical and structural features, such as URL length, the number of dots or hyphens, presence of digits, or whether the URL contains suspicious tokens. Algorithms such as Logistic Regression, Support Vector Machines, Random Forests, and Decision Trees demonstrated strong baseline performance [3]. These models offer interpretability and low computational overhead, making them suitable candidates for deployment in operational security tools, though they may struggle against sophisticated obfuscation and rapidly evolving phishing techniques.

More recently, deep learning approaches, including character-level convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Transformer-based models, have been applied to URL classification, automatically learning representations without manual feature engineering [8]. These models have demonstrated strong performance by capturing complex lexical patterns and contextual relationships within URL strings. However, their deployment in real-time or computationally constrained environments is often limited by computational requirements, memory usage, inference latency, and reduced interpretability [8]. For environments where explainability, low overhead, and offline operation are required, lightweight interpretable classifiers remain the more appropriate choice [9].

A growing body of research highlights a key challenge that affects both lightweight and deep learning approaches equally: generalization across datasets. Models trained on one dataset often exhibit significant performance degradation when tested on another dataset collected from a different time period, geographic region, source platform, or threat intelligence feed. This phenomenon, broadly referred to as dataset shift or domain shift, reveals inherent differences in URL distribution, phishing techniques, and labeling practices across sources [10]. As phishing campaigns evolve rapidly, and threat intelligence feeds capture different segments of the malicious URL ecosystem, a model's performance on a single dataset is not a reliable indicator of real-world deployment performance.

This challenge is particularly relevant when comparing phishing-specific feeds such as PhishTank, which captures credential-harvesting URLs, typically characterized by long obfuscated paths and deceptive domain structures, against malware delivery feeds such as URLhaus, which tracks botnet infrastructure URLs often characterized by IP-based hosts, higher proportions of numeric characters, and structural patterns distinct from traditional phishing [4], [5]. Models trained on one type of threat may not generalize well to the other, and quantifying this asymmetry is essential for understanding the practical limitations of URL-only detection [10], [11].

An additional constraint that has received comparatively little attention in the literature is the requirement to operate without any external data sources at inference time. Many published phishing detection systems that demonstrate strong performance incorporate external supplementary signals. While these supplementary signals improve accuracy, they introduce dependencies that are incompatible with privacy-sensitive, air-gapped, or resource-constrained deployment environments. The design space of fully offline, URL-string-only detection systems capable of maintaining strong performance without these signals remains underexplored relative to its practical deployment relevance [9], [11].

This project situates itself within this evolving research landscape by focusing on lightweight, interpretable ML classifiers operating under a strict URL-string-only feature constraint, evaluated against two independently sourced real-world threat intelligence datasets. Rather than pursuing deep learning accuracy, the project emphasizes methods that are computationally efficient, fully offline, and suitable for operational deployment [9]. The cross-dataset generalization framework directly addresses the dataset shift problem by training on one threat source and evaluating on the other in both directions, providing quantitative evidence of how well URL-structural patterns transfer across fundamentally different categories of malicious URLs, and establishing a reproducible foundation for evaluating offline phishing detection under realistic operational constraints.